PLAN MAESTRO DE ASEGURAMIENTO DE CALIDAD DE SOFTWARE (SQAP v3.0)

Sistema de Gestión Nocturna "Discoteca Tío Chu" — Marco normativo para los Sprints 1 a 4

Documento: SQAP-Maestro-v3.0
Versión: 3.0 — Plan Maestro congelado, marco normativo de todo el ciclo de vida (Sprints 1 a 4)
Institución: Universidad San Francisco Xavier de Chuquisaca (USFX)
Facultad: Ciencia y Tecnología
Carrera: Ciencias de la Computación
Asignatura: COM450 — Calidad de Software
Docente: Ing. Carlos Montellano
Gestión: II/2026
Lugar y fecha: Sucre, Bolivia — octubre de 2026
Equipo: EQ-LAB07

1. Propósito

Este Plan Maestro de Aseguramiento de Calidad de Software (SQAP v3.0) establece las actividades, responsabilidades, estándares, métricas, revisiones, pruebas, controles y evidencias que se aplicarán a todo el ciclo de vida del Sistema de Gestión Nocturna "Discoteca Tío Chu", es decir, a los Sprints 1, 2, 3 y 4 y a cualquier iteración posterior. Es el marco normativo congelado del proyecto: define qué se mide, con qué umbrales, con qué herramientas y bajo qué responsabilidades, y no registra resultados de ejecución. Su finalidad es permitir demostrar, mediante resultados reproducibles conservados en los Informes de Cierre de Sprint, que las funcionalidades entregadas cumplen los requisitos acordados y que los riesgos de errores en sesiones, inventario, cobros, comisiones, nómina, liquidación y cierres se mantienen bajo control.

El plan abarca la revisión de requisitos, el diseño, la implementación, las pruebas unitarias y de integración, las pruebas de sistema, la revisión de código, el control de cambios y la auditoría de salida de cada sprint.

Este plan adopta la estructura de IEEE Std 730-2002 (16 secciones y anexos) y el modelo de calidad ISO/IEC 25010 para evaluar las características clave del software.

1.1 Alcance funcional

Este Plan Maestro rige las 17 historias de la Pila de Producto (Backlog v5), organizadas en 4 sprints. La matriz siguiente es la línea base de trazabilidad completa; el estado corresponde a la fecha de emisión de la v3.0 y su valor vigente se mantiene en el Backlog y en los Informes de Cierre de Sprint:

ID	Historia	Puntos	ROI	Prioridad	Estado	Depende de
Sprint 1 (Completadas · 18 pts · ROI 1.50) — Datos maestros, apertura de sesión e inventario inicial						
h2	Apertura de Sesión	3	2.67	Alta	Completada	—
h11	Ajuste Stock Bodega	5	1.20	Media	Completada	—
h12	Despacho a Barras	5	1.40	Media	Completada	h11, h2
h15	Maestro de Personal	5	1.20	Media	Completada	—
Sprint 2 (Completadas · 20 pts · ROI 1.55) — Configuración del turno y ventas en vivo						
h14	Cobros QR	3	2.67	Alta	Completada	h2
h13	Retención de Tarjetas POS	4	1.75	Media	Completada	h2
h6	Carga Nómina Diaria	5	1.40	Media	Completada	h2, h15
h4	Conteo Botellas/Combos	8	1.13	Alta	Completada	h12, h2
Sprint 3 (Planificadas · 21 pts · ROI 1.57) — Liquidación, pagos y cierres operativos						
h5	Liquidación por Turno	3	3.33	Alta	Planificada	h4, h13, h14, h6
h3	Stock Inicial Barra	5	1.80	Alta	Planificada	h4, h12

ID	Historia	Puntos	ROI	Prioridad	Estado	Depende de
h16	Pago Rápido Personal	5	1.60	Media	Planificada	h6, h15
h17	Pago Deudas Atrasadas	8	0.75	Baja	Planificada	h16, h15
Sprint 4 (Planificadas · 20 pts · ROI 2.00) — Balance final, arqueo y dashboard						
h7	Balance Express	3	3.33	Alta	Planificada	h13, h14, h4, h5, h16, h17
h8	Arqueo Efectivo Físico	3	3.00	Alta	Planificada	h5, h16, h17
h9	Dashboard Ejecutivo	3	2.67	Alta	Planificada	h4, h13, h14, h8
h18	Cierre Seguro de Noche	3	2.67	Alta	Planificada	h2, h7, h8
h10	Vista Táctil Bodega	8	0.63	Baja	Planificada	h11
Total	**17 historias · 4 sprints**	**79**	**1.66**	**—**	**—**	**ROI = Puntos Valor ÷ Puntos Historia (131 ÷ 79)**

Toda historia incorporada posteriormente al Backlog queda cubierta, sin modificar este plan, por las métricas permanentes de la sección 5. La columna “Depende de” determina el alcance de la regresión: al cerrar cada sprint se reejecutan los casos críticos de las historias de las que depende el incremento (por ejemplo, `h5` depende de `h4`, `h13`, `h14` y `h6`). Las cifras de puntos y ROI provienen del Backlog v5 y no se recalculan en este plan.

1.2 Clasificación de integridad y criticidad

El proyecto adopta **Nivel de Integridad 2 — Moderado** como clasificación de gestión de riesgos para todo el sistema: una falla puede producir sesiones mal abiertas o cerradas, inconsistencias de inventario, diferencias de caja, registros de cobro incorrectos, ventas reales, liquidaciones, balances o pagos de jornal mal calculados, pero el sistema no controla directamente funciones de seguridad física. Esta etiqueta se utiliza como decisión del proyecto y no como una certificación independiente de cumplimiento de IEEE Std 1012.

Se consideran especialmente críticos para la calidad:

- `h2` y `h18`: evitan operaciones vinculadas a una sesión nocturna incorrecta o inexistente y ediciones accidentales de una noche ya sellada.
- `h11`, `h12`, `h3` y `h10`: controlan que entradas, mermas, ajustes, despachos a barras y saldos de apertura no generen cantidades incoherentes ni pérdidas de trazabilidad entre bodega y barras.
- `h4`: evita diferencias entre el conteo de botellas sobrantes, los combos descontados y las ventas reales, rompiendo la conciliación del inventario.
- `h13` y `h14`: evitan errores aritméticos, de redondeo, duplicaciones y proveedores o montos inválidos en los cobros con tarjeta y por QR.
- `h5`, `h8`, `h7` y `h9`: evitan cifras financieras incoherentes en la liquidación por turno, el arqueo de efectivo físico, el balance y el dashboard ejecutivo.
- `h15`, `h6`, `h16` y `h17`: evitan registros de personal duplicados o con cargos fuera del catálogo normalizado, nóminas del día incorrectas y pagos de jornal erróneos, duplicados o atrasados sin trazabilidad.

1.3 Objetivos de aseguramiento de calidad

1. Ejecutar el 100 % de los casos de prueba definidos como obligatorios para las historias del sprint en curso y reejecutar los casos críticos de regresión de las historias de las que dependen.
2. Alcanzar una cobertura de líneas de al menos 80 % en el código incluido en el alcance y 90 % en los componentes críticos medibles.
3. Mantener cero defectos S1 y S2 abiertos al cierre de cada sprint.
4. Impedir la integración a `main` de cambios sin revisión independiente y sin el pipeline obligatorio aprobado.
5. Conservar evidencia identificable de pruebas, inspecciones, análisis estático, defectos y decisiones de aceptación.
6. Verificar el rendimiento con un escenario de carga reproducible; el resultado no se considerará aprobado si no se conserva el informe de JMeter.
7. Automatizar con Cypress al menos un escenario E2E crítico por cada historia del Backlog (100 % de las historias) y conservar el reporte de ejecución, las capturas y los videos de los fallos en el Informe de Cierre del sprint.
8. Verificar el estilo PSR-12 con Laravel Pint en el hook pre-commit (Husky) y repetir la verificación en CI, sin incumplimientos.
9. Registrar los resultados reales de cada iteración exclusivamente en los Informes de Cierre de Sprint, de modo que este Plan Maestro permanezca sin modificaciones por resultados durante los Sprints 3 y 4.

2. Documentos de referencia

Código	Documento	Versión/edición	Uso en este SQAP
SRS	Especificación de Requisitos de Software de `tio_chu`	Vigente	Fuente de requisitos funcionales, restricciones y criterios de aceptación de las historias de cada sprint.
SAD	Documento de Arquitectura de Software de `tio_chu`	Vigente	Fuente de decisiones arquitectónicas, componentes, persistencia y límites entre módulos.
PB	`Historias_Usuario_Tio_Chu_v5_4_Sprints.xlsx`	v5 (4 sprints)	Fuente del backlog, identificadores, puntos, ROI, prioridad y distribución de sprints.
ICS	`Informes de Cierre de Sprint (docs/pruebas/informe-sprintX.md, X = 1 a 4)`	Uno por sprint	Registro independiente de los resultados reales de cada iteración; no modifica este plan.
IEEE 730	IEEE Std 730-2002, *Standard for Software Quality Assurance Plans*	Edición 2002	Referencia para organizar las actividades de aseguramiento de calidad.
ISO 25010	ISO/IEC 25010:2011, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*	Edición 2011	Modelo de características de calidad utilizado para organizar las métricas.
IEEE 1028	IEEE Std 1028-2008, *Standard for Software Reviews and Audits*	Edición 2008	Referencia para revisiones, inspecciones y auditoría de cierre.
PSR-12	PHP-FIG, PSR-12: Extended Coding Style Guide	Versión publicada por PHP-FIG	Convenciones de estilo PHP, aplicadas mediante Laravel Pint.
Laravel	Documentación de Laravel 13	Versión objetivo del proyecto	Patrones de controladores, validación, servicios, migraciones, pruebas y configuración.
Pest/PHPUnit	Documentación de Pest PHP y PHPUnit	Versión compatible con el entorno	Pruebas automatizadas unitarias y de integración.
JMeter	Apache JMeter	5.6	Ejecución de escenarios de carga y recopilación de resultados.
GitHub Flow	Política de ramas y Pull Requests del repositorio	Aplicación del equipo	Revisión independiente, protección de `main` y trazabilidad de cambios.

Control de configuración documental: los documentos SRS y SAD vigentes de tio_chu se consideran referencias requeridas por este plan. Si aún no están almacenados y aprobados en el repositorio, su estado deberá registrarse como pendiente de control documental; este SQAP no afirma que existan archivos aprobados sin evidencia.

3. Gestión

El EQOO7 asume los diversos roles del proyecto garantizando la separacion de funciones mediante revisiones independientes y el uso de herramientas automatizadas para asegurar la calidad de cada sprint.

3.1 Organización, autoridad y responsabilidades

Actividad/rol	Maturano Zárate Félix	Clarós André
Liderazgo y coordinación del sprint	Responsable	Consultado
Arquitectura y decisiones técnicas	Responsable	Consultado/implementador
Responsable SQA principal	Revisor secundario y apoyo	Responsable
Desarrollo backend/frontend	Puede implementar tareas asignadas	Responsable principal de implementación
Diseño y ejecución de pruebas	Tester Lead; verifica casos críticos	Implementa y ejecuta pruebas automatizadas
Revisión independiente de PR creados por André	Obligatoria	No puede aprobar su propio PR
Revisión de artefactos elaborados por Félix	Revisión cruzada por André; para artefactos SQA, revisión adicional por checklist automatizable	Responsable de revisión documental
Clasificación y seguimiento de defectos	Confirma severidad en defectos críticos	Registra, diagnostica y corrige defectos asignados
Aprobación técnica de integración	Aprueba tras comprobar criterios	Solicita integración; no se autoaprueba
Auditoría de cierre	Coauditor y comprobador de evidencias	Responsable de consolidar evidencias

3.2 Regla de independencia de SQA

La regla de independencia de SQA establece que **ningún integrante puede aprobar ni fusionar su propio trabajo**, garantizando la objetividad mediante **revisiones cruzadas** entre los miembros del equipo:

- **Separación estricta entre autor y aprobador:** André no aprueba sus propios Pull Requests ni métricas; Félix los revisa y valida antes del cierre (y viceversa cuando Félix sea el autor).
- **Criterios de aprobación:** Toda validación exige cumplir con una lista de chequeo, pruebas ejecutadas y análisis estático de código.
- **Transparencia y escalamiento:** Al ser un equipo de dos personas existe una limitación estructural, por lo que ante decisiones arquitectónicas o de evaluación complejas se recurre a la revisión del docente.

3.3 Matriz RACI

R: ejecuta la actividad. **A:** autoridad que responde por el resultado. **C:** consultado. **I:** informado. Se asigna una sola `A` por fila.

Actividad de calidad	Félix	André
Elaborar y actualizar el SQAP	C	A/R
Revisar y aprobar la línea base de requisitos SRS	A/R	R
Inspeccionar el SAD y registrar hallazgos	A/R	R
Revisar PR cuyo autor sea André	A/R	R (autor, no aprobador)
Revisar PR cuyo autor sea Félix	R (autor, no aprobador)	A/R
Mantener pruebas unitarias, de integración y E2E (Cypress)	C	A/R
Verificar de manera independiente los casos críticos	A/R	R
Ejecutar y analizar pruebas JMeter	A	R
Clasificar y priorizar defectos críticos	A/R	R
Mantener los registros y evidencias	C	A/R
Ejecutar auditoría de cierre del sprint	A/R	R
Emitir recomendación técnica de aceptación	A/R	C
Aceptación académica o autorización del docente, si aplica	C	C

Cuando la misma fila parezca atribuir `R` al autor de un cambio y `A/R` al revisor, la responsabilidad `R` del autor se refiere a preparar el artefacto y atender observaciones; la autoridad de aprobación sigue correspondiendo exclusivamente al revisor designado.

4. Documentación

Los artefactos se mantendrán versionados. Las fechas indicadas por semana son hitos relativos al sprint en curso; no sustituyen la fecha real de creación que debe constar en Git.

Artefacto	Responsable de elaboración	Hito	Tipo de revisión	Criterio de adecuación	Evidencia
SRS	André; Félix revisa	Antes de implementar las historias	Revisión de requisitos	Las historias del sprint tienen precondiciones, flujo principal, validaciones, excepciones y criterios de aceptación medibles.	`docs/srs/ (SRS vigente)` y PR de aprobación
SAD	Félix, con aportes de André	Antes de integrar módulos	Inspección técnica/Fagan adaptada	Arquitectura coherente con Laravel, MySQL y los casos de uso del sprint; se identifican persistencia, validaciones y transacciones relevantes.	`docs/arquitectura/ (SAD vigente)`, checklist y acta
SQAP Maestro v3.0	André	Emisión y ante cambios normativos	Revisión documental y auditoría SQA	16 secciones, 3 anexos, métricas instrumentadas, roles asignados y evidencias trazables.	`docs/calidad/SQAP-Maestro-v3.0.docx`
Manual de usuario (uno por sprint)	André; Félix prueba instrucciones	Antes de la demostración	Revisión de usabilidad	Describe las funciones entregadas hasta el sprint evaluado con pasos reproducibles; no documenta funciones futuras como si estuvieran disponibles.	`docs/manual/usuario-sprintX.md`

Artefacto	Responsable de elaboración	Hito	Tipo de revisión	Criterio de adecuación	Evidencia
Plan y casos de prueba	André; Félix revisa casos críticos	Antes de la ejecución formal	Revisión técnica	Cada caso tiene ID, requisito, datos, pasos, resultado esperado y resultado obtenido.	`tests/`, `cypress/e2e/` y `docs/pruebas/`
Informe de Cierre de Sprint	André; Félix verifica	Cierre del sprint	Auditoría de resultados	Registra de forma independiente los resultados reales del sprint: aprobados/fallidos, cobertura, métricas obtenidas, defectos pendientes, rendimiento, evidencias de Cypress y JMeter, desviaciones y limitaciones de ejecución.	`docs/pruebas/informe-sprintX.md` y artefactos CI (incluye reportes y capturas de Cypress)
Actas SQA	André registra; Félix valida	Cada revisión formal	Revisión de acta	Fecha, asistentes, versión revisada, hallazgos, responsable y decisión documentados.	`docs/actas/`
Registro de defectos	André registra; Félix confirma S1/S2	Continuo	Revisión de defectos	Pasos de reproducción, severidad, prioridad, evidencia, responsable, corrección y revalidación.	GitHub Issues o `docs/defectos/`

4.1 Desacoplamiento de evidencias: Informes de Cierre de Sprint

Este Plan Maestro es un marco normativo congelado: contiene umbrales, métodos, herramientas, responsables y criterios de aceptación, pero no contiene resultados de ejecución. Los resultados numéricos reales de las pruebas, las métricas obtenidas, las capturas y videos de Cypress, los archivos de JMeter (.jmx, .jtl y reporte HTML) y las actas de aceptación de cada iteración se registran de manera independiente en el Informe de Cierre de Sprint correspondiente (por ejemplo, docs/pruebas/informe-sprint1.md, informe-sprint2.md, informe-sprint3.md e informe-sprint4.md).

En consecuencia, la ejecución de los Sprints 3 y 4 no requiere modificar este documento: un resultado desfavorable genera un defecto, un riesgo o una desviación en el informe del sprint, nunca una edición del plan. El plan solo cambia mediante el procedimiento de la sección 16 ante un cambio normativo aprobado. Cada métrica del informe se identifica con su ID de la sección 5.2 y se compara con el umbral allí definido; el informe no redefine umbrales.

Cada Informe de Cierre de Sprint contendrá como mínimo:

- **Identificación:** sprint, historias evaluadas según la matriz 1.1, commit o tag y entorno de ejecución.
- **Pruebas y cobertura:** resultados de Pest/PHPUnit y cobertura global y crítica (M-01, M-02), con el reporte adjunto.
- **Pruebas E2E:** escenarios críticos por historia ejecutados con cypress run, reporte, capturas y videos de los fallos (M-14, M-15).
- **Rendimiento:** archivos .jmx y .jtl, reporte HTML de JMeter 5.6, p95 y throughput con 50 usuarios concurrentes (M-03, M-04).
- **Calidad de código:** resultados de SonarQube, Laravel Pint (PSR-12) y revisión de dependencias (M-05, M-06, M-16).
- **Defectos y desviaciones:** registro de defectos, S1/S2 abiertos (criterio M-17) y desviaciones o limitaciones de ejecución.
- **Aceptación:** acta del sprint (docs/actas/acta-sprintX.md) aprobada por una persona distinta del autor.

5. Estándares, prácticas, convenciones y métricas

5.1 Convenciones de desarrollo

Elemento	Convención
Estilo PHP	PSR-12, verificado con Laravel Pint (preset psr12) en el hook pre-commit mediante Husky y repetido en CI.
Controllers y Models	PascalCase; por ejemplo, `NightSessionController`, `QrPaymentController`.
Servicios	PascalCase con sufijo descriptivo; por ejemplo, `StockBodegaService`.
Métodos y variables PHP	camelCase; por ejemplo, `calculateNetAmount()`.
Tablas y columnas MySQL	snake_case en minúsculas; tablas en plural cuando corresponda al modelo de datos; por ejemplo, `night_sessions`, `created_at`.
Migraciones	Convención Laravel y nombres que describan la operación y tabla afectada.
Rutas REST	URI en minúsculas, sustantivos y guiones cuando sean necesarios; por ejemplo, `/api/night-sessions` y `/api/qr-payments`.
Validación	Validación en servidor para todos los datos de entrada; no confiar únicamente en la interfaz.

Elemento	Convención
Errores	Respuestas consistentes, sin exponer secretos, trazas internas o datos sensibles.
Persistencia	Usar transacciones cuando una operación modifique de forma coordinada existencias de bodega y barra, o el conteo de cierre y el descuento de combos.
Pruebas	Nombres que describan el comportamiento esperado, la condición y el resultado.
Pruebas E2E (Cypress)	Archivos *.cy.js en cypress/e2e, uno por historia (por ejemplo, h14-cobros-qr.cy.js); selectores mediante atributos data-cy; datos sintéticos reiniciables por escenario; sin credenciales ni datos reales.
Configuración	Secretos mediante variables de entorno; nunca incorporar credenciales al repositorio.

La arquitectura defensiva podrá usar `Schema::hasTable` o `Schema::hasColumn` únicamente cuando el contexto de despliegue o migración lo justifique. Estos chequeos no reemplazan las migraciones versionadas ni deben ocultar errores estructurales de base de datos.

5.2 Matriz de métricas de calidad

Los umbrales de esta tabla son **objetivos normativos internos del SQAP**. Deben comprobarse con el código y las herramientas efectivamente instaladas. Si una herramienta o integración aún no está configurada, la métrica se marcará como “no instrumentada” y no como aprobada.

ID	Característica ISO/IEC 25010	Métrica y fórmula	Umbral de aceptación	Instrumento y condiciones	Responsable
M-01	Mantenibilidad	Cobertura de líneas = líneas ejecutadas / líneas ejecutables × 100	≥ 80 % global del código incluido	Pest/PHPUnit con driver de cobertura compatible; pipeline CI	André; Félix verifica
M-02	Mantenibilidad	Cobertura en componentes críticos	≥ 90 % en la lógica crítica medible (cálculos financieros, inventario, sesión nocturna, liquidación y pagos; ver 1.2)	Reporte de cobertura filtrado por clases/servicios críticos	André; Félix verifica
M-03	Eficiencia de desempeño	Percentil 95 del tiempo de respuesta HTTP	p95 ≤ 1.5 s en endpoints del alcance bajo la carga definida	JMeter; 50 usuarios concurrentes, 10 min de carga sostenida, entorno documentado	André ejecuta; Félix revisa
M-04	Eficiencia de desempeño	Throughput = solicitudes completadas / segundo	≥ 30 solicitudes/s en el escenario de carga acordado, si el entorno permite medirlo de forma válida	JMeter, sin errores funcionales y con entorno identificado	André; Félix verifica
M-05	Seguridad	Vulnerabilidades críticas detectadas en código nuevo	0 críticas; 0 altas sin plan de tratamiento aprobado	SonarQube/SonarCloud y escaneo de dependencias configurado	André; Félix revisa
M-06	Mantenibilidad	Duplicación de líneas	< 3 % en código nuevo	SonarQube en cada PR	André; Félix revisa
M-07	Usabilidad	System Usability Scale (SUS)	Media ≥ 68/100	Cuestionario SUS aplicado a participantes que ejecuten tareas de cada sprint; registrar n y resultados	Félix coordina; André registra
M-08	Adecuación funcional	Tasa de aprobación de pruebas obligatorias = pruebas aprobadas / pruebas ejecutadas × 100	100 % de casos críticos y ≥ 95 % de todos los casos; ningún S1/S2 abierto	Pest, pruebas HTTP, Cypress e informe de ejecución	André; Félix verifica
M-09	Fiabilidad / integridad de datos	Operaciones de inventario con saldo consistente	100 % de casos probados conservan la igualdad entre movimiento registrado y saldos resultantes, incluida la conciliación tras el conteo de cierre	Pruebas de integración contra MySQL; casos de transferencia, conteo de cierre, descuento de combos, cantidad insuficiente y rollback	André; Félix verifica
M-10	Adecuación funcional	Exactitud de los cálculos de negocio (neto de tarjeta, ventas reales por combos, liquidación por turno, balance, arqueo y planilla de jornales)	100 % de casos de prueba coinciden con las fórmulas y reglas de redondeo aprobadas en el SRS	Pruebas unitarias con valores límite, decimales y tasas o reglas configuradas	André; Félix verifica
M-11	Adecuación funcional	Exactitud de identificación de día de sesión y de nómina diaria	100 % de casos de fecha y de carga de nómina producen el día de semana y el personal activo programado esperados	Pruebas unitarias y de integración con fechas representativas, personal activo e inactivo y zona horaria definida	André; Félix verifica
M-12	Seguridad / integridad	Registros duplicados por repetición de una solicitud (apertura de sesión, alta de trabajador, cobro QR, carga o pago de nómina, cierre de noche)	0 duplicados en los escenarios de repetición contemplados por los requisitos	Pruebas de integración para clave de idempotencia o regla de unicidad, si está implementada	André; Félix verifica

ID	Característica ISO/IEC 25010	Métrica y fórmula	Umbral de aceptación	Instrumento y condiciones	Responsable
M-13	Adecuación funcional	Exactitud de la conciliación de inventario por conteo (combos, botellas extra y saldo de apertura)	100 % de casos de prueba producen los combos vendidos, las botellas extra y los saldos esperados, incluida la sincronización del stock inicial de la noche siguiente	Pruebas unitarias e integración con composiciones de combos y conteos sintéticos, incluidos valores límite	André; Félix verifica
M-14	Adecuación funcional	Tasa de aprobación de escenarios E2E críticos = escenarios Cypress críticos aprobados / ejecutados × 100	100 % de escenarios críticos aprobados y 0 escenarios inestables sin tratamiento documentado	Cypress (cypress run, modo headless) en CI sobre entorno de pruebas con datos sintéticos; reporte, capturas y videos de fallos archivados	André implementa; Félix verifica
M-15	Adecuación funcional	Cobertura de historias por E2E = historias del Backlog con al menos un escenario crítico Cypress aprobado / historias del Backlog × 100	100 % de las historias del Backlog vigente (17 de 17 en la línea base de esta versión), cada una con al menos 1 escenario crítico ejecutado con cypress run	Trazabilidad TC-H*-nn (caso E2E de cada historia) ↔ especificaciones en cypress/e2e; informe de ejecución de Cypress	Félix coordina; André registra
M-16	Mantenibilidad	Cumplimiento de estilo PSR-12 = archivos PHP sin incumplimientos / archivos PHP verificados × 100	100 % (0 incumplimientos en cada commit y en CI)	Laravel Pint (pint --test) en pre-commit con Husky; verificación repetida en el pipeline de CI	André; Félix verifica
M-17	Fiabilidad / integridad de datos	Defectos S1/S2 abiertos = incidencias S1 y S2 sin cierre verificado	0 al cierre de cada sprint y al integrar a main	Registro de defectos (GitHub Issues o docs/defectos/) y Quality Gate	André registra; Félix confirma

Definiciones operativas para evitar ambigüedades

- **Exactitud de cálculos:** Resultados de comisión, neto, combos, liquidación, balance, arqueo y planilla de jornales sujetos a las fórmulas y reglas de redondeo aprobadas en el SRS y a datos de prueba acordados.
- **Idempotencia:** Validación contra la repetición de una misma solicitud mediante identificador único de transacción o regla de unicidad definida en el SRS; si no existe, se documenta la falta de protección contra duplicados.
- **Conciliación de combos:** Verificación de que el stock de apertura, el conteo de botellas sobrantes y los combos descontados no dejan diferencia residual, según la composición de combos aprobada en el SRS.
- **Escenario E2E crítico:** Flujo de interfaz de una historia del Backlog automatizado con Cypress y ejecutado en CI con datos sintéticos; su fallo bloquea el merge.
- **Cobertura (Global / Crítica):** Porcentaje de código probado delimitado pre-ejecución al alcance real del *release*, excluyendo código generado o migraciones.
- **p95 / TPS:** Métricas de rendimiento (percentil 95 y solicitudes por segundo) reportadas con transparencia según los límites del entorno local.
- **SUS:** Puntuación estándar de usabilidad obtenida exclusivamente con participantes y datos reales.

5.3 Homologación de umbrales y configuración

Los valores del SQAP y las configuraciones ejecutables deben mantenerse sincronizados. Los fragmentos siguientes son una **línea base de configuración propuesta**, no una afirmación de que ya existan en el repositorio. Al integrarlos, el equipo verificará versiones y sintaxis contra el proyecto Laravel/Pest instalado.

- **`phpunit.xml`:** Establece la configuración estándar para la ejecución de pruebas unitarias y la generación del reporte de cobertura de código.
- **`pest.php`:** Define el contexto base para la suite de pruebas (Unitarias y de Integración), delegando la verificación de los umbrales de cobertura a la herramienta de CI.
- **`sonar-project.properties`:** Identifica el proyecto, rutas de código fuente, pruebas e informes de cobertura para el análisis en SonarQube/SonarCloud, dejando la validación de métricas al *Quality Gate* del servidor o del pipeline.

- **`cypress.config.js`: Define el baseUrl del entorno de pruebas, la ubicación de las especificaciones (cypress/e2e), los reintentos permitidos y el almacenamiento de capturas y videos de los fallos, dejando la verificación de resultados al job de CI.**
- **`.husky/pre-commit`: Ejecuta Laravel Pint en modo de verificación (pint --test) antes de cada commit; pint.json fija el preset psr12 y el pipeline de CI repite la verificación, porque un hook local puede omitirse.**

Métrica	Umbral SQAP	Configuración/ejecutor requerido	Criterio de comprobación
Cobertura global	≥ 80 %	Generación Clover/XML en CI y chequeo de umbral; SonarQube opcional como segundo control	El pipeline falla si la cobertura es inferior al 80 %.
Cobertura crítica	≥ 90 %	Reporte filtrado para los componentes críticos y chequeo específico en CI	El informe identifica archivos incluidos y porcentaje.
p95	≤ 1.5 s	Plan JMeter versionado, con assertion de duración/percentil analizado en el informe	Informe con carga, tiempos y errores; el p95 medido cumple el límite.
Throughput	≥ 30 req/s	JMeter y reporte agregado	Solicitudes exitosas por segundo ≥ 30 en el escenario especificado.
Vulnerabilidades críticas	0	Quality Gate y escaneo de dependencias habilitados	Ninguna vulnerabilidad crítica abierta en el código nuevo; las alertas se documentan.
Duplicación	< 3 %	Quality Gate SonarQube	El indicador de duplicación de código nuevo es inferior al 3 %.
Pruebas obligatorias	100 % aprobadas	Pipeline Pest/PHPUnit	Cualquier fallo de un caso crítico bloquea el merge.
Escenarios E2E críticos	100 % de historias con ≥ 1 escenario crítico aprobado	Job de CI con cypress run (modo headless) y reporte archivado como artefacto	Cualquier fallo de un escenario crítico bloquea el merge; el informe identifica escenario, historia y resultado, y ninguna historia queda sin escenario.
Estilo PSR-12	0 incumplimientos	Laravel Pint (pint --test) en pre-commit (Husky) y en el pipeline de CI	El commit y el PR se bloquean ante cualquier incumplimiento.
Defectos S1/S2 abiertos	0	Registro de defectos y Quality Gate	Cualquier S1/S2 abierto bloquea la integración a main y el cierre del sprint.

Regla de homologación: si se modifica un umbral, se actualizarán simultáneamente este SQAP, la configuración automatizada, los casos de prueba afectados y el historial de cambios. No se considerará suficiente cambiar únicamente el documento.

6. Revisiones de software

Se planifican revisiones estructuradas con base en las prácticas de revisión y auditoría de IEEE Std 1028-2008. El formato exacto se adaptará al tamaño del equipo, manteniendo registro de participantes, hallazgos, decisiones y cierre.

Tipo	Artefacto	Momento	Responsable de revisión	Criterio de aceptación	Evidencia
Revisión de requisitos	SRS vigente y criterios de las historias del sprint	Antes de desarrollo y ante cambios	Félix lidera; André presenta	Cada historia tiene flujo, validaciones, excepciones y resultados esperados sin ambigüedades críticas.	Checklist y acta
Inspección técnica tipo Fagan	SAD vigente y decisiones de diseño	Antes de integrar lógica crítica	Félix modera; André participa	Sin hallazgos críticos o mayores abiertos que afecten sesión, inventario, cobros, liquidación, nómina o cierre de noche.	Registro de defectos del documento
Revisión de Pull Request	Código, pruebas, migraciones y configuración	En cada PR	Revisor distinto del autor	Checklist completo, pruebas aprobadas, análisis estático conforme y sin secretos ni cambios no relacionados.	PR aprobado y ejecución CI
Revisión de pruebas	Casos, datos y resultados	Antes y después de ejecutar	Félix verifica casos críticos; André mantiene suite	Trazabilidad a requisitos y evidencia de resultado real.	Informe de Cierre de Sprint
Auditoría SQA de cierre	SQAP, código, informes, defectos y manual	Al finalizar cada sprint	Félix coordina; André consolida	Siete indicadores de verificabilidad evaluados; cero S1/S2 abiertos; desviaciones documentadas.	Acta de cierre

6.1 Checklist obligatorio de Pull Request

☐ El PR está vinculado a una historia del Backlog.

☐ El cambio tiene alcance acotado y descripción de la solución.
☐ Se añadieron o actualizaron pruebas automatizadas.
☐ Los escenarios E2E de Cypress afectados se añadieron o actualizaron y usan solo datos sintéticos.
☐ Las pruebas relacionadas pasan.
☐ Laravel Pint (PSR-12) pasa en el hook pre-commit y en CI sin incumplimientos.
☐ Se revisaron validaciones de entrada, errores y autorización, cuando corresponda.
☐ Se evaluó la consistencia transaccional en operaciones de inventario.
☐ No se incorporaron credenciales, datos bancarios sensibles, datos personales reales del personal ni archivos locales.
☐ El Quality Gate y el pipeline están aprobados, si están configurados.
☐ La aprobación corresponde a una persona distinta del autor.
☐ La evidencia de revisión queda enlazada al PR o al acta.

7. Pruebas

7.1 Niveles de prueba

Nivel	Técnica/herramienta	Responsable	Criterio de entrada	Criterio de salida
Unitario	Pest/PHPUnit; mocks para dependencias externas	André	Reglas de negocio identificadas y pruebas reproducibles	100 % de pruebas unitarias obligatorias aprobadas; cobertura según M-01 y M-02.
Integración	Laravel HTTP Tests contra MySQL de pruebas; factories y transacciones de prueba	André; Félix revisa casos críticos	Migraciones aplicables ejecutadas y ambiente de prueba aislado	Sin errores de persistencia, contratos API o consistencia de inventario.
Sistema/E2E	Cypress (cypress run en CI): al menos un escenario crítico por cada historia del Backlog; pruebas HTTP/API complementarias y regresión de los flujos críticos de las historias de las que depende el incremento	Félix coordina; André implementa los escenarios Cypress	Build integrado en entorno de prueba, aplicación accesible por baseUrl y datos sintéticos preparados	Todos los escenarios E2E críticos aprobados en cypress run; evidencia (reporte, capturas y videos de fallos) archivada; ningún S1/S2 abierto.
Carga	Apache JMeter 5.6	André ejecuta; Félix revisa el informe	Ambiente identificado, datos controlados y endpoints estables	p95 ≤ 1.5 s y throughput ≥ 30 req/s en el escenario acordado, o desviación documentada como no conformidad.
Aceptación funcional	Demostración basada en criterios de las historias del sprint	Félix dirige; André documenta	Pruebas de sistema terminadas	Criterios de aceptación de las historias del sprint satisfechos o pendientes claramente registrados; cero S1/S2 abiertos.

Las pruebas E2E se automatizan con Cypress y se ejecutan en CI contra el entorno de pruebas. Si el entorno de interfaz o la configuración de Cypress aún no existen, la métrica correspondiente se marcará como “no instrumentada”, se ejecutará una prueba de sistema manual estructurada y se registrará como tal; no se presentará como E2E automatizada.

7.2 Casos de prueba mínimos por historia

Los casos siguientes son el mínimo obligatorio por historia del Backlog. Cada historia incluye un caso E2E ejecutado con Cypress; los demás casos se ejecutan con Pest/PHPUnit y pruebas HTTP. Los casos de cada historia se ejecutan en el sprint que la entrega y se reejecutan como regresión mientras otras historias dependan de ella (columna “Depende de” de 1.1). Los resultados no se registran aquí sino en el Informe de Cierre de Sprint.

`h2` — Apertura de sesión nocturna (Sprint 1)

ID	Escenario	Resultado esperado
TC-H2-01	Abrir una sesión con una fecha válida	Se guarda una sesión con la fecha indicada y el día de semana correcto; quedan habilitadas las operaciones de venta y arqueo de esa fecha.
TC-H2-02	Abrir sesión en una fecha límite de mes/año	El día de semana coincide con la fecha de calendario; no hay desplazamiento por zona horaria.
TC-H2-03	Intentar una apertura duplicada para la misma fecha, si la regla de negocio la prohíbe	Se rechaza o gestiona conforme al requisito, sin duplicación silenciosa.
TC-H2-04	Enviar fecha vacía o inválida	Validación controlada; no se crea registro inválido.

ID	Escenario	Resultado esperado
TC-H2-05	E2E (Cypress): seleccionar una fecha y abrir la sesión desde la interfaz	La interfaz muestra la sesión abierta con la fecha y el día de semana correctos; el escenario aprueba en cypress run.

`h11` — Ajuste de stock en bodega central (Sprint 1)

ID	Escenario	Resultado esperado
TC-H11-01	Incrementar stock con los botones +1 y +10	El saldo aumenta exactamente la cantidad solicitada y se guarda sin recargar la página.
TC-H11-02	Decrementar stock con los botones -1 y -10 con saldo suficiente	El saldo disminuye exactamente la cantidad solicitada.
TC-H11-03	Intentar dejar stock negativo	Se impide la operación o se aplica la regla aprobada; no queda saldo negativo accidental.
TC-H11-04	Editar el stock directamente con guardado automático	El nuevo saldo se guarda automáticamente, coincide con el valor ingresado y no requiere recargar la página; un valor no numérico o negativo se rechaza con mensaje controlado.
TC-H11-05	Registrar una entrada y una merma	Se actualiza el saldo y queda trazabilidad de cantidad y tipo de movimiento.
TC-H11-06	Error de persistencia durante el ajuste	No queda una actualización parcial; la transacción se revierte cuando corresponda.
TC-H11-07	E2E (Cypress): ajustar el stock con un botón de paso y por edición directa	El saldo mostrado se actualiza y persiste sin recargar la página; el escenario aprueba en cypress run.

`h12` — Apertura de inventario en barras, despacho (Sprint 1)

ID	Escenario	Resultado esperado
TC-H12-01	Registrar cajas y botellas hacia una barra con stock suficiente en bodega central	Se registra salida de bodega y entrada a la barra con cantidades coherentes; queda fijado el stock de apertura de esa barra.
TC-H12-02	Registrar la apertura de varias barras o puntos de despacho	Cada barra queda correctamente identificada; no se mezclan saldos entre barras.
TC-H12-03	Solicitar cantidad mayor que el saldo de bodega	Se rechaza el despacho sin dejar cantidades negativas ni registros parciales.
TC-H12-04	Registrar la apertura con el rol Bartender y con el rol Administrador	El registro se completa con ambos roles y los saldos resultantes son idénticos para los mismos datos de entrada.
TC-H12-05	Forzar error durante la transferencia	La operación se revierte o queda en un estado recuperable y auditable; no se pierde stock.
TC-H12-06	Consultar inventario tras el despacho	La variación de bodega y la variación de la barra corresponden al movimiento registrado.
TC-H12-07	E2E (Cypress): registrar la apertura de una barra desde la interfaz	El stock de apertura mostrado coincide con las cajas y botellas registradas; el escenario aprueba en cypress run.

`h15` — Gestión de nómina y personal (Sprint 1)

ID	Escenario	Resultado esperado
TC-H15-01	Registrar un trabajador con nombre, celular, cargo válido y jornal habitual	El trabajador se guarda con todos sus datos y aparece en la plantilla centralizada.
TC-H15-02	Registrar un trabajador por cada cargo normalizado (MESERO, BARTENDER, SEGURIDAD, LIMPIEZA)	Cada cargo se almacena en su forma normalizada, sin variantes de escritura.
TC-H15-03	Ingresar un cargo que no pertenece al catálogo normalizado	Se rechaza con mensaje controlado; no se crea ni modifica el registro.
TC-H15-04	Enviar nombre vacío, celular con formato inválido (según la regla del SRS) o jornal cero, negativo o no numérico	Validación en servidor con mensaje controlado; no se crea registro inválido.
TC-H15-05	Intentar registrar un trabajador duplicado, si la regla de negocio lo prohíbe	Se rechaza o gestiona conforme al requisito, sin duplicación silenciosa.
TC-H15-06	Modificar los datos de un trabajador existente y consultar la plantilla	Los cambios se guardan, la plantilla muestra los datos actualizados y no se alteran otros registros.
TC-H15-07	E2E (Cypress): registrar un trabajador y consultarlo en la plantilla	El trabajador aparece con su cargo normalizado y su jornal habitual; el escenario aprueba en cypress run.

`h14` — Cobros QR con selección de proveedor (Sprint 2)

ID	Escenario	Resultado esperado
TC-H14-01	Registrar un cobro QR seleccionando el proveedor Yasta con monto válido	Se guarda el proveedor correcto, el monto y los datos requeridos, asociados a la sesión nocturna abierta.
TC-H14-02	Registrar un cobro QR seleccionando el proveedor Yape con monto válido	Se guarda el proveedor correcto, el monto y los datos requeridos, asociados a la sesión nocturna abierta.
TC-H14-03	Intentar registrar un cobro QR sin seleccionar proveedor	Se rechaza con mensaje controlado; no se crea el cobro.
TC-H14-04	Enviar monto cero, negativo o formato inválido	La solicitud se rechaza con mensaje controlado y no altera los cobros registrados.
TC-H14-05	Repetir una solicitud con el mismo identificador, si existe mecanismo de idempotencia	No se duplica el cobro. Si no existe tal mecanismo, se registra la limitación como riesgo.
TC-H14-06	Registrar un cobro QR sin una sesión nocturna abierta (dependencia de h2)	Se rechaza con mensaje controlado; el cobro no queda asociado a una sesión inexistente.
TC-H14-07	E2E (Cypress): seleccionar el proveedor, ingresar el monto y confirmar el cobro desde la interfaz	El cobro se muestra con el proveedor y el monto correctos; el escenario aprueba en cypress run.

`h13` — Comisión y neto de tarjeta POS (Sprint 2)

ID	Escenario	Resultado esperado
TC-H13-01	Registrar un voucher con importe bruto válido y tasa configurada	Comisión y neto coinciden con la fórmula aprobada.
TC-H13-02	Importe con decimales	Redondeo aplicado de forma consistente con la regla documentada.
TC-H13-03	Importe cero, negativo o tasa fuera del rango permitido	Se rechaza o maneja según la regla de negocio; no se calcula un neto inválido.
TC-H13-04	Valores límite y tasa del 0 % en ambiente de prueba	No hay desbordamiento ni pérdida de precisión inesperada.
TC-H13-05	Consultar el registro tras guardar el voucher	Se conservan el importe bruto, la comisión y el neto, y se cumple bruto = comisión + neto.
TC-H13-06	E2E (Cypress): ingresar el voucher en la interfaz y confirmar el cálculo automático	La comisión y el neto mostrados se calculan automáticamente y coinciden con el cálculo de referencia; el escenario aprueba en cypress run.

Fórmula de referencia, si la comisión se expresa como porcentaje: `comisión = redondear(bruto × tasa / 100)` y `neto = bruto − comisión`. La regla exacta de redondeo y la tasa aplicable deben estar aprobadas en SRS/configuración antes de usar los resultados como aceptación bancaria real.

`h6` — Carga automática de nómina al abrir la noche (Sprint 2)

ID	Escenario	Resultado esperado
TC-H6-01	Abrir la noche con personal activo programado para el día de la sesión	La nómina del día se carga automáticamente con los trabajadores activos programados y su jornal pactado, sin reescribir la lista.
TC-H6-02	Abrir la noche con trabajadores inactivos o no programados para ese día	Los trabajadores inactivos o no programados no aparecen en la nómina cargada.
TC-H6-03	Abrir sesiones en distintos días de la semana y en una fecha límite de mes/año	Cada nómina corresponde al día de la semana de la sesión; no hay desplazamiento por zona horaria.
TC-H6-04	Abrir la noche sin personal activo programado para el día	Se carga una nómina vacía con mensaje controlado; la apertura no falla.
TC-H6-05	Reintentar la carga o reabrir la misma sesión, si la regla de negocio prohíbe duplicar la nómina	No se duplica la nómina del día; no hay registros repetidos de un mismo trabajador.
TC-H6-06	Forzar un error de persistencia durante la carga de la nómina	No queda una nómina parcial; la operación se revierte o queda en un estado recuperable y auditable.
TC-H6-07	E2E (Cypress): abrir una noche y consultar la nómina cargada	La nómina visible coincide con el personal activo programado para el día; el escenario aprueba en cypress run.

h4 — Conteo de botellas sobrantes y combos (Sprint 2)

ID	Escenario	Resultado esperado
TC-H4-01	Registrar el conteo de botellas sobrantes en una barra con stock de apertura	El conteo se guarda asociado a la barra y a la sesión; no modifica el stock de apertura registrado.
TC-H4-02	Calcular combos vendidos y botellas extra con una composición de combos aprobada	Los combos vendidos y las botellas extra coinciden con el cálculo de referencia; el descuento de combos se refleja en el inventario.
TC-H4-03	Ingresar un conteo mayor que el stock de apertura, negativo o no numérico	Se rechaza con mensaje controlado; no se registra el conteo ni se descuentan combos.
TC-H4-04	Registrar un conteo igual al stock de apertura y un conteo cero	Se calculan consumo cero y consumo total, respectivamente, sin errores ni saldos negativos.
TC-H4-05	Verificar la conciliación del inventario tras el conteo	Stock de apertura = botellas sobrantes + botellas de combos vendidos + botellas extra; no queda diferencia residual.
TC-H4-06	Registrar el conteo de varias barras en la misma noche	Cada barra se concilia de forma independiente; no se mezclan saldos entre barras.
TC-H4-07	Forzar un error de persistencia durante el descuento de combos	La operación se revierte o queda en un estado recuperable y auditable; no se pierde ni se duplica stock.
TC-H4-08	E2E (Cypress): registrar el conteo de cierre de una barra y revisar combos y botellas extra	Los valores mostrados coinciden con el cálculo de referencia y la conciliación no presenta diferencia; el escenario aprueba en cypress run.

Regla de referencia para la conciliación: `consumo = stock de apertura − botellas sobrantes` y `consumo = botellas de combos vendidos + botellas extra`. La composición exacta de cada combo debe estar aprobada en SRS/datos de prueba antes de usar los resultados como aceptación de ventas reales.

h5 — Liquidación por turno (Sprint 3)

ID	Escenario	Resultado esperado
TC-H5-01	Consultar la liquidación de un turno con ventas (h4), cobros QR (h14) y tarjetas (h13) registrados	El cuadro muestra el total vendido en bebidas, lo cobrado por QR, lo tarjeteado y el efectivo neto por bartender, coincidiendo con el cálculo de referencia.
TC-H5-02	Verificar el efectivo neto de cada bartender con la fórmula aprobada en el SRS	El efectivo neto coincide con la fórmula aprobada, sin diferencias de redondeo.
TC-H5-03	Liquidar un turno sin cobros QR ni tarjetas	El efectivo neto corresponde al total vendido; no hay errores.
TC-H5-04	Liquidar un turno con varios bartenders de la nómina del día (h6)	Cada bartender se liquida de forma independiente y solo aparecen los de la nómina del día.
TC-H5-05	Consultar la liquidación de un turno sin ventas registradas	Los valores se muestran en cero con mensaje controlado.
TC-H5-06	E2E (Cypress): abrir la liquidación de un turno con datos sintéticos de ventas, QR y tarjetas	Las cifras mostradas coinciden con el cálculo de referencia; el escenario aprueba en cypress run.

h3 — Stock inicial de barra (Sprint 3)

ID	Escenario	Resultado esperado
TC-H3-01	Sincronizar el stock de apertura con los saldos finales de la noche anterior de una barra	Los saldos de apertura coinciden con los saldos finales (h4) de la noche anterior, sin recontar ni digitar.
TC-H3-02	Sincronizar varias barras en la misma noche	Cada barra copia únicamente su propio saldo final.
TC-H3-03	Sincronizar cuando no existe noche anterior (primera noche)	No se copia ningún saldo y se conserva el stock de apertura fijado con h12.
TC-H3-04	Repetir la sincronización sobre la misma sesión, si la regla de negocio lo prohíbe	No se duplica ni se altera el stock ya sincronizado.
TC-H3-05	Forzar un error de persistencia durante la sincronización	No queda stock parcial; la operación se revierte o queda en un estado recuperable y auditable.
TC-H3-06	E2E (Cypress): sincronizar el inventario de apertura desde la interfaz	Los saldos mostrados coinciden con los saldos finales de la noche anterior; el escenario aprueba en cypress run.

h16 — Pago rápido de personal (Sprint 3)

ID	Escenario	Resultado esperado
TC-H16-01	Marcar como pagado a un trabajador de la nómina del día	El estado queda pagado y persistido; la planilla se recalcula de inmediato.
TC-H16-02	Editar el jornal de un trabajador de la nómina del día	El total de la planilla se recalcula en el DOM sin recargar y coincide con el total persistido en servidor.
TC-H16-03	Editar el jornal con valor cero, negativo o no numérico	Se rechaza con mensaje controlado; la planilla no cambia.
TC-H16-04	Repetir la solicitud de pago de un mismo trabajador	No se duplica el pago ni el egreso asociado.
TC-H16-05	Comparar el total mostrado en pantalla con el total persistido tras varias ediciones	Ambos totales coinciden exactamente.
TC-H16-06	E2E (Cypress): marcar un pago y editar un jornal desde la interfaz	La planilla se recalcula de inmediato y el resultado persiste; el escenario aprueba en cypress run.

h17 — Pago de deudas atrasadas (Sprint 3)

ID	Escenario	Resultado esperado
TC-H17-01	Pagar con “Pagar Jornal” el jornal atrasado de un trabajador	La deuda de la noche pasada queda liquidada y la interfaz se actualiza sin recargar la página.
TC-H17-02	Repetir “Pagar Jornal” sobre un jornal ya pagado	No se duplica el pago ni el egreso.
TC-H17-03	Simular un fallo de la solicitud asíncrona (red o servidor)	La interfaz informa el error y no marca el jornal como pagado; no queda un pago parcial.
TC-H17-04	Consultar el egreso tras pagar un jornal atrasado	El egreso queda registrado y disponible para el balance (h7) y el arqueo (h8).
TC-H17-05	E2E (Cypress): pagar un jornal atrasado desde la interfaz	El estado cambia a pagado sin recargar la página; el escenario aprueba en cypress run.

h7 — Balance express (Sprint 4)

ID	Escenario	Resultado esperado
TC-H7-01	Consultar el balance con ingresos por tarjetas, QR y efectivo y egresos por pagos	Los totales coinciden con el cálculo de referencia (ingresos − egresos). Los gastos se incluyen cuando exista una historia que los registre.
TC-H7-02	Verificar que el balance cabe en una sola pantalla sin scroll en la resolución de referencia del SRS	No aparece scroll vertical ni horizontal en el viewport de referencia.
TC-H7-03	Consultar el balance de una noche sin egresos y de una sin ingresos	Los valores son correctos y no hay errores.
TC-H7-04	Contrastar el balance con las liquidaciones (h5) y los pagos (h16, h17) registrados	Los ingresos y egresos del balance coinciden con los registros de origen.
TC-H7-05	E2E (Cypress): abrir el balance de una noche con datos sintéticos	Los totales mostrados coinciden con el cálculo de referencia; el escenario aprueba en cypress run.

h8 — Arqueo de efectivo físico (Sprint 4)

ID	Escenario	Resultado esperado
TC-H8-01	Calcular el dinero físico neto esperado con el efectivo liquidado (h5) y los pagos al personal (h16, h17)	El resultado coincide con la fórmula aprobada en el SRS.
TC-H8-02	Verificar la clasificación del resultado como Plata a Favor o Faltante	La clasificación corresponde a la regla del SRS y el signo del resultado es correcto.
TC-H8-03	Calcular el arqueo de una noche sin movimientos de efectivo	El resultado es cero y no hay errores.
TC-H8-04	Calcular el arqueo tras pagar jornales atrasados (h17)	Los pagos atrasados se reflejan como egresos en el resultado.
TC-H8-05	E2E (Cypress): abrir el arqueo de una noche con datos sintéticos	El resultado mostrado coincide con el cálculo de referencia; el escenario aprueba en cypress run.

`h9` — Dashboard ejecutivo (Sprint 4)

ID	Escenario	Resultado esperado
TC-H9-01	Consultar los cuatro indicadores (Venta en Barras, Tarjetas Neto Banco, Cobros QR y Efectivo en Caja) de una noche	Cada indicador coincide con su origen: h4, h13, h14 y h8, respectivamente.
TC-H9-02	Modificar un dato fuente (por ejemplo, registrar un cobro QR) y volver a consultar	El indicador correspondiente cambia; no hay valores simulados ni estáticos.
TC-H9-03	Consultar el dashboard de una noche sin datos	Los indicadores se muestran en cero con mensaje controlado.
TC-H9-04	Medir la carga del dashboard en el escenario de carga	Se cumple p95 ≤ 1.5 s (M-03).
TC-H9-05	E2E (Cypress): abrir el dashboard de una noche con datos sintéticos	Los cuatro indicadores mostrados coinciden con los datos de origen; el escenario aprueba en cypress run.

`h18` — Cierre seguro de noche (Sprint 4)

ID	Escenario	Resultado esperado
TC-H18-01	Sellar una noche con balance (h7) y arqueo (h8) realizados	La noche queda en estado cerrado.
TC-H18-02	Intentar editar cobros, stock o pagos de una noche sellada	La edición se bloquea con mensaje controlado y no hay cambios.
TC-H18-03	Reabrir una noche sellada	Se permiten las correcciones y la reapertura queda registrada, según el requisito del SRS.
TC-H18-04	Reabrir con un rol no autorizado, si el requisito restringe la acción	Se rechaza con mensaje controlado; la noche permanece sellada.
TC-H18-05	Repetir la solicitud de cierre sobre una noche ya sellada	No se altera el estado ni se duplican registros.
TC-H18-06	E2E (Cypress): sellar una noche, comprobar el bloqueo y reabrirla	El bloqueo y la reapertura se reflejan en la interfaz; el escenario aprueba en cypress run.

`h10` — Vista táctil de bodega (Sprint 4)

ID	Escenario	Resultado esperado
TC-H10-01	Alternar entre la Tabla Ejecutiva y la Cuadrícula de Tarjetas	Ambas vistas muestran los mismos datos y saldos.
TC-H10-02	Ajustar el stock desde la cuadrícula táctil	El saldo persiste y coincide con el resultado de h11.
TC-H10-03	Cargar más de 30 marcas en cada vista	Todas las marcas se muestran sin pérdida de datos y con tiempos de respuesta dentro de M-03.
TC-H10-04	Operar los controles en el viewport táctil de referencia del SRS	Los controles son operables sin errores de interacción.
TC-H10-05	E2E (Cypress): alternar entre las dos vistas y ajustar un saldo	La alternancia y el ajuste funcionan; el escenario aprueba en cypress run.

7.3 Escenario de carga

Plan JMeter propuesto para la línea base:

- 50 usuarios concurrentes.
- 2 minutos de ramp-up, salvo que la capacidad del entorno requiera otra configuración justificada.
- 10 minutos de carga sostenida.
- Ejecución sobre endpoints representativos de las historias del sprint evaluado y de los flujos críticos acumulados, en un ambiente no productivo.
- Registro de tasa de error, p50, p95, p99, throughput, número de solicitudes y configuración del servidor.
- Datos de prueba no sensibles; no usar transferencias reales, vouchers reales, datos personales reales del personal ni credenciales bancarias.
- Repetir la prueba después de una corrección relevante de rendimiento.

Los objetivos de p95 y throughput solo serán comparables entre ejecuciones si se conserva la configuración del entorno, versión del código, datos y plan de prueba.

7.4 Criterios globales de entrada y salida

Entrada: SRS revisado; historias priorizadas; entorno de prueba documentado; migraciones disponibles; build instalable; casos de prueba revisados; datos sintéticos preparados.

Salida: ejecución de todos los casos obligatorios; cobertura evaluada; resultados JMeter registrados; defectos clasificados; cero defectos S1/S2 abiertos; informe de cierre revisado y evidencia archivada. Un requisito no instrumentado no se considerará aprobado por ausencia de medición.

8. Reporte de problemas y acción correctiva

8.1 Flujo de estados

`Nuevo → Asignado → En resolución → Resuelto → En verificación → Cerrado`

Si la corrección falla en la revalidación: `En verificación → Reabierto → En resolución`.

Cada defecto debe incluir identificador, título, historia afectada, versión/build, ambiente, pasos de reproducción, resultado esperado, resultado obtenido, evidencia, severidad, prioridad, responsable, causa raíz, corrección y prueba de regresión.

La **severidad** mide el impacto técnico y operativo; la **prioridad** expresa la urgencia de corrección. SQA propone la severidad y el Líder de Proyecto confirma la prioridad. En un equipo de dos personas, los defectos S1/S2 deben revisarse conjuntamente.

8.2 Severidad y SLA

Los tiempos se cuentan desde el registro y la notificación del defecto durante el periodo de trabajo acordado. Son objetivos internos de respuesta, no garantías de atención 24/7.

Severidad	Definición	Respuesta máxima	Objetivo de resolución	Tratamiento
S1 — Crítica	Pérdida/corrupción de datos de sesión, cobro, nómina o inventario; cálculo del neto de tarjeta, liquidación, balance, arqueo o saldos que produce importes incorrectos con impacto financiero; o imposibilidad general de abrir, operar o cerrar la noche sin alternativa segura.	< 2 h	< 24 h	Bloquea la release; análisis inmediato, corrección y regresión independiente.
S2 — Alta	Función central de una historia inutilizable o con resultados incorrectos sin alternativa operativa aceptable.	< 4 h laborables	< 48 h laborables	Bloquea la release hasta corregir y verificar.
S3 — Media	Funcionalidad secundaria afectada con alternativa segura y sin corrupción de datos.	< 1 día laborable	< 5 días laborables o antes del cierre del sprint	Se corrige antes de liberar si afecta aceptación; de lo contrario se registra decisión.
S4 — Baja	Defecto visual, texto o inconsistencia menor sin impacto sobre datos ni flujo crítico.	< 2 días laborables	Siguiente iteración o fecha acordada	Se prioriza según valor y esfuerzo; no se oculta en el informe.

8.3 Acción correctiva y prevención de recurrencia

1. Reproducir el defecto con datos controlados.
2. Identificar causa raíz, distinguiéndola del síntoma.
3. Crear una prueba que falle antes de la corrección cuando sea viable.
4. Implementar el arreglo en una rama específica.
5. Ejecutar pruebas unitarias, integración y regresión relacionadas.
6. Obtener aprobación de un revisor distinto del autor.
7. Actualizar el registro, vincular evidencia y cerrar solo después de la verificación.
8. Si el defecto se repite, revisar el diseño, el requisito o el proceso de revisión, no solo volver a parchear el mismo punto.

9. Herramientas, técnicas y metodologías

Herramienta	Uso previsto	Evidencia de ejecución	Condición de aceptación
Git / GitHub	Historial de cambios, ramas, PR e incidencias	Commits, PR y Issues	Trazabilidad entre historia, cambio y revisión.
GitHub Actions	Integración continua	Logs y artefactos del workflow	Jobs obligatorios aprobados antes de merge.
Laravel Pint	Formato y estilo PHP (PSR-12)	Salida del comando y CI	Sin incumplimientos de formato no justificados.
Husky	Hook pre-commit de Git: ejecuta Laravel Pint (pint --test) antes de cada commit	Configuración .husky/pre-commit y salida del hook	Commit bloqueado ante incumplimientos PSR-12; el CI repite la verificación.
Pest PHP / PHPUnit	Pruebas unitarias e integración	Reportes de ejecución	Cero fallos en casos críticos; cobertura conforme.

Herramienta	Uso previsto	Evidencia de ejecución	Condición de aceptación
Cypress	Pruebas end-to-end automatizadas de interfaz (cypress run en CI)	Especificaciones en cypress/e2e, reporte de ejecución, capturas y videos de fallos	Escenarios críticos aprobados; métricas M-14/M-15 evaluadas; versión fijada en package-lock.json.
PHPStan / Larastan	Análisis estático de tipos y errores potenciales	Log/versiones/configuración	Nivel acordado y errores críticos tratados; el nivel efectivo debe declararse.
SonarQube/SonarCloud	Calidad estática, duplicación y seguridad	Quality Gate y reporte	Umbrales M-05/M-06 cumplidos o no conformidad registrada.
Apache JMeter 5.6	Rendimiento y concurrencia	`.jmx`, `.jtl` y reporte HTML	Resultados comparables y objetivos M-03/M-04 evaluados.
MySQL 8.0	Persistencia de pruebas	Versión, esquema, logs y fixtures	Integridad y consistencia verificadas.
Dependabot / OWASP Dependency-Check	Revisión de dependencias	Alertas e informes	Vulnerabilidades altas/críticas atendidas dentro del SLA.

9.1 Prácticas adoptadas

- **TDD obligatorio** en los cálculos de negocio (comisión y neto de tarjeta, combos y conciliación, liquidación, balance, arqueo y planilla de jornales), en la validación de montos y en las transferencias de inventario: primero se define la prueba de comportamiento, después se implementa la lógica y finalmente se refactoriza manteniendo la suite en verde.
- **Revisión de PR por otra persona:** no se permite autoaprobación.
- **Refactorización defensiva:** validar entradas, cantidades, estados y errores de persistencia; los chequeos de esquema se usan de manera explícita y no como sustituto de migraciones correctas.
- **Pruebas de regresión:** toda corrección de un defecto S1/S2 añade o actualiza pruebas que prevengan su reaparición.
- **Pruebas E2E con Cypress:** cada historia del Backlog cuenta con al menos un escenario crítico automatizado que se ejecuta en CI; los escenarios usan datos sintéticos y selectores data-cy estables, y un escenario inestable se trata como defecto de prueba, no se oculta con reintentos indefinidos.
- **Configuración reproducible:** las versiones de PHP, Laravel, MySQL y dependencias se registran en el repositorio y en el Informe de Cierre de Sprint.
- **Pre-commit obligatorio:** el hook de Husky ejecuta Laravel Pint (PSR-12) antes de cada commit; al ser omisible localmente, el pipeline de CI repite la verificación y bloquea el merge.
- **Sin evidencia ficticia:** un job no ejecutado, un resultado no medido o una herramienta aún no conectada se declara como pendiente.

10. Control de medios

10.1 GitHub Flow y protección de `main`

- `main` contiene la línea estable del proyecto.
- El desarrollo se realiza en ramas `feature/*`, `fix/*` o `hotfix/*`.
- Se prohíbe el push directo a `main`.
- Cada cambio se integra mediante Pull Request.
- Los PR requieren aprobación de una persona distinta del autor.
- Deben pasar las pruebas, el análisis estático y los chequeos de calidad que estén configurados como obligatorios.
- No se permitirá que una excepción de pipeline se use para ocultar una falla crítica. Cualquier bypass excepcional deberá justificarse y quedar en acta; un S1/S2 abierto bloquea la release.
- Los cambios en migraciones, archivos de configuración, workflows y dependencias también requieren revisión.

10.2 Copias de seguridad

Elemento	Frecuencia	Retención	Responsable	Prueba de restauración
Código fuente y documentación	Historial continuo en GitHub; copia de respaldo semanal mediante `git bundle`	Historial del repositorio y copias de los últimos 3 meses	André; Félix comprueba la restauración	Clonar/restaurar en entorno temporal una vez al mes.
Base de datos MySQL de pruebas	Dump SQL diario al finalizar la jornada de trabajo	30 días en almacenamiento seguro separado	André; Félix verifica evidencia	Restauración mensual en una base aislada; comparar tablas y conteos relevantes.
Evidencias de CI y pruebas	En cada pipeline y release	Al menos las últimas 30 ejecuciones; informes de cierre conservados durante el proyecto	André	Comprobar que los enlaces y artefactos son accesibles al auditar.

Las copias no deben garantizar la confidencialidad, excluyendo credenciales y datos sensibles para proteger la informacion del proyecto.

11. Control de proveedores

11.1 Política de selección

Las dependencias de Composer y npm se evaluarán antes de incorporarse o actualizarse considerando:

1. Licencia compatible con el uso académico y previsto: preferencia por MIT, BSD y Apache 2.0, previa comprobación de la licencia concreta.
2. Mantenimiento reciente, idealmente con actividad en los últimos 12 meses, o justificación de estabilidad si el paquete está maduro y sin mantenimiento activo.
3. Compatibilidad con las versiones de PHP, Laravel, Node.js y herramientas del proyecto.
4. Ausencia de vulnerabilidades críticas conocidas sin mitigación.
5. Reputación, documentación, comunidad y posibilidad de sustituir el paquete.
6. Mínimo privilegio y ausencia de scripts o permisos innecesarios.

No se afirmará que todas las dependencias actuales cumplen estos criterios hasta ejecutar y revisar el inventario real de `composer.lock` y `package-lock.json`.

11.2 Escaneo y respuesta

- Dependabot se habilitará para Composer y npm si el repositorio lo permite.
- OWASP Dependency-Check u otra herramienta compatible se ejecutará semanalmente y al actualizar dependencias.
- Alertas altas o críticas: evaluación y plan de actualización en menos de 48 horas.
- Si no existe parche, se documentará la exposición, el impacto, la mitigación temporal y la fecha de revisión.
- Se conservará evidencia de la versión vulnerable, versión corregida, pruebas de regresión y PR de actualización.
- La licencia se comprobará antes de aceptar una dependencia nueva.

12. Registros: recolección, mantenimiento y retención

Evidencia	Ubicación	Formato	Retención	Responsable
Actas de revisión e inspección	`docs/actas/`	Markdown/PDF	Toda la vida del proyecto	André; validación de Félix
Casos de prueba	`docs/pruebas/` y `tests/`	Markdown/PHP	Toda la vida del proyecto	André
Reportes de Pest/PHPUnit	Artefactos CI y docs/pruebas/informe-sprintX.md	XML/HTML	Últimas 30 ejecuciones; reporte final permanente en el proyecto	André
Informe de Cierre de Sprint	docs/pruebas/informe-sprintX.md	Markdown/PDF	Toda la vida del proyecto	André; revisión de Félix
Reportes de Cypress (E2E)	Artefactos CI y docs/pruebas/cypress/sprintX/ (reporte, capturas y videos de fallos)	JSON/HTML, PNG, MP4	Últimas 30 ejecuciones; reporte final permanente en el proyecto	André; revisión de Félix
Reportes de cobertura	Artefactos CI	Clover/XML/HTML	Últimas 30 ejecuciones; reporte final permanente	André
Reportes JMeter	`docs/pruebas/jmeter/sprintX/`	`.jmx`, `.jtl`, HTML	Toda la vida del proyecto para pruebas de release	André; revisión de Félix
Reportes SonarQube	Dashboard y enlace en el Informe de Cierre de Sprint	Dashboard/exportación	Al menos 6 meses si la plataforma lo permite; conservar evidencia de cierre	André
Pull Requests y revisiones	GitHub	Registro de PR	Toda la vida del repositorio	Ambos
Defectos	GitHub Issues o `docs/defectos/`	Issue/Markdown	Toda la vida del proyecto	André; Félix confirma S1/S2
Copias de base de datos	Ubicación de respaldo con acceso restringido	Dump SQL comprimido	30 días	André
Acta de aceptación de cada sprint	`docs/actas/acta-sprintX.md`	Markdown/PDF	Toda la vida del proyecto	Félix y André

Los registros deben nombrarse con fecha, versión y contexto asegurando la trazabilidad mediante el commit, entorno, configuración de ejecución y resultado. Los resultados reales de cada iteración se conservan exclusivamente en estos registros y en el Informe de Cierre de Sprint (sección 4.1); este plan no los replica.

13. Capacitación

La capacitación se orienta a cerrar brechas necesarias para ejecutar el plan. Las sesiones son actividades planificadas, no se consideran completadas hasta registrar fecha, contenido y resultado.

Competencia	Integrante	Brecha/objetivo	Actividad planificada	Evidencia
GitHub Actions y Quality Gates	Ambos	Asegurar que las pruebas y el análisis estático bloqueen PRs incorrectos	Taller interno de 2 horas para revisar workflows, fallos y artefactos	Acta, workflow y ejecución demostrativa
Pest/PHPUnit y TDD	André; Félix como revisor	Crear pruebas primero para la lógica de cálculo e inventario	Sesión práctica sobre casos límite, mocks y pruebas de integración	PR con pruebas y checklist
JMeter	André; Félix revisa	Construir una prueba reproducible con concurrencia y lectura correcta de p95/TPS	Laboratorio de 2 horas: Thread Group, ramp-up, listeners y reporte	Archivo `.jmx` e informe
Cypress (E2E)	André; Félix como revisor	Automatizar escenarios E2E con selectores estables, datos sintéticos y ejecución headless en CI	Sesión práctica: escribir y ejecutar con cypress run los escenarios críticos de las historias de cada sprint	Especificaciones en PR y reporte de ejecución
PHPStan/Larastan, Pint y Husky	André	Aplicar estilo consistente y detectar errores antes de revisión manual	Ejecución local, hook pre-commit y revisión de resultados en CI	Logs y configuración versionada
Inspección técnica y revisión independiente	Ambos	Mantener separación entre autor y aprobador	Simulación de revisión de PR y registro de hallazgos	Acta de inspección
Integridad de inventario y transacciones	Ambos	Comprender efectos de concurrencia, rollback y consistencia de saldos	Revisión de casos de transferencia y fallos de persistencia	Casos de prueba revisados

14. Gestión de riesgos

14.1 Método de evaluación

Se adopta una matriz de Probabilidad e Impacto con escala de 1 a 3:

- **Probabilidad (P):** 1 = baja, 2 = media, 3 = alta.
- **Impacto (I):** 1 = bajo, 2 = medio, 3 = alto.
- **Exposición:** (E = P \times I).
- **Bajo:** 1–2; **Medio:** 3–4; **Alto:** 6–9.

Todo riesgo alto requiere mitigación accionable, disparador definido, responsable y contingencia. Los valores son estimaciones iniciales del equipo y se revisarán en cada sprint (resultados en el Informe de Cierre) con evidencia.

14.2 Registro de riesgos

ID	Descripción	P	I	E / nivel	Mitigación accionable	Responsable	Disparador (trigger)	Contingencia
R-01	Cobertura inferior a los umbrales o falta de pruebas en lógica crítica.	2	3	6 — Alto	TDD en neto, combos e inventario; añadir pruebas límite; bloquear PR si falla el umbral configurado.	André; Félix verifica	Cobertura global <80 % o crítica <90 %.	Priorizar pruebas de lógica crítica y posponer cambios no esenciales.
R-02	Rendimiento insuficiente durante concurrencia de atención.	2	3	6 — Alto	Medir con JMeter, revisar consultas SQL, índices y transacciones; conservar baseline.	André ejecuta; Félix revisa	p95 >1.5 s o throughput <30 req/s.	Identificar endpoints lentos, optimizar y repetir prueba; registrar limitación si el entorno invalida la medición.
R-03	Error de configuración o fallo del pipeline de CI.	2	2	4 — Medio	Validar workflow en rama de prueba y mantener pasos reproducibles.	André	Dos ejecuciones consecutivas fallan por configuración.	Ejecutar comprobaciones locales, corregir pipeline y mantener bloqueada la integración de cambios no verificados.
R-04	Vulnerabilidad crítica/alta en dependencia Composer o npm.	2	3	6 — Alto	Activar Dependabot y escaneo semanal; revisar licencias y lockfiles.	André; Félix verifica	Alerta alta/crítica nueva.	Actualizar o sustituir la dependencia; si no hay parche, aislar función afectada y documentar riesgo aceptado por escrito.
R-05	Pérdida de código, documentación o evidencia por fallo local.	1	3	3 — Medio	Push frecuente, copia semanal `git bundle`, backup diario de DB y restauración mensual.	André; Félix comprueba	No se puede recuperar un commit o dump reciente.	Restaurar desde copia verificada y registrar incidente; reconstruir evidencia desde CI cuando sea posible.
R-06	Ambigüedad en las reglas de comisión, redondeo, composición de combos o programación de personal por día.	3	3	9 — Alto	Revisar criterios de aceptación antes de codificar; acordar tasa, redondeo, composición de combos y regla de programación de personal con datos de prueba explícitos.	Félix coordina; André documenta	Dos interpretaciones distintas o resultados esperados sin definición.	Suspender aceptación de la funcionalidad afectada hasta aprobar la regla y actualizar SRS/casos de prueba.
R-07	Inconsistencia o pérdida de stock entre bodega y barras, o descuadre de la conciliación de combos tras el conteo de cierre.	2	3	6 — Alto	Operaciones transaccionales, validación de saldo, pruebas de cantidades insuficientes y rollback.	André; Félix verifica	Diferencia entre movimiento registrado y saldos o saldo negativo inesperado.	Bloquear nuevos despachos afectados, reconciliar movimientos con evidencia y aplicar corrección auditada.
R-08	Ausencia temporal de uno de los dos integrantes o concentración de conocimiento.	2	3	6 — Alto	Documentar decisiones, usar PR pequeños, revisión por pares y mantener instrucciones de ejecución actualizadas.	Ambos	Tarea crítica sin responsable disponible o bloqueo superior a un día laborable.	Repriorizar el sprint, limitar alcance a funciones verificables y solicitar orientación docente si compromete la entrega.

ID	Descripción	P	I	E / nivel	Mitigación accionable	Responsable	Disparador (trigger)	Contingencia
R-09	Regresión en historias entregadas en sprints anteriores de las que dependen las historias del sprint en curso (columna “Depende de” de 1.1).	2	3	6 — Alto	Mantener en CI las pruebas críticas de las historias entregadas como suite de regresión y vincular cada PR con las dependencias afectadas.	André; Félix verifica	Falla de un caso crítico de una historia entregada en una ejecución de CI.	Bloquear la integración del PR, corregir la regresión con prueba previa y repetir la suite completa.
R-10	Inestabilidad o falta de entorno para las pruebas E2E con Cypress (escenarios intermitentes, datos no reproducibles).	2	2	4 — Medio	Usar datos sintéticos reiniciables por escenario, selectores data-cy estables y esperas basadas en aserciones; fijar la versión de Cypress en package-lock.json.	André; Félix verifica	Un escenario crítico falla y aprueba sin cambios de código en ejecuciones consecutivas.	Tratar el escenario como defecto de prueba; entre tanto, ejecutar el flujo como prueba de sistema manual estructurada, registrada como tal y no como E2E automatizada.
R-11	Requisitos sin historia asociada en el Backlog v5: gastos como egreso de h7 y h8, catálogo de productos y combos para h4 y h11, y roles y control de acceso.	3	3	9 — Alto	Solicitar a Don Ludo y al equipo historias o requisitos explícitos en el SRS antes de iniciar las historias dependientes; registrar supuestos y datos de prueba.	Félix coordina; André documenta	Una historia dependiente (h4, h7, h8, h11 o control de acceso) entra al sprint sin criterio de aceptación para el requisito faltante.	No declarar aprobada la historia en los aspectos no definidos; registrar la desviación en el Informe de Cierre y proponer el cambio de alcance (sección 16).
R-12	Carga de un sprint superior a la capacidad supuesta de ~20 puntos de historia (Sprint 3 con 21 puntos), con riesgo de recortar pruebas para cumplir el alcance.	2	3	6 — Alto	Diferir primero h10 (sin dependientes, ROI 0.63, prioridad Baja) en lugar de recortar pruebas; ajustar la carga a la velocidad real medida en el informe del sprint anterior.	Félix coordina; André ejecuta	Velocidad real del sprint anterior inferior a los puntos comprometidos.	Replanificar el alcance del sprint por el procedimiento de la sección 16; mantener intactos los umbrales de calidad.

15. Glosario

Término	Definición
SQAP	*Software Quality Assurance Plan*: plan que define actividades, responsabilidades, estándares y evidencias de aseguramiento de calidad.
SRS	*Software Requirements Specification*: especificación de requisitos funcionales y no funcionales del sistema.
SAD	*Software Architecture Document*: documento de arquitectura que describe componentes, decisiones técnicas e interacciones.
ISO/IEC 25010	Modelo de calidad de sistemas y software utilizado para organizar características y subcaracterísticas de calidad.
IEEE 730	Estándar de referencia para estructurar planes de aseguramiento de calidad de software.
IEEE 1028	Estándar de referencia para revisiones e inspecciones de software y auditorías.
p95	Percentil 95 del tiempo de respuesta; el 95 % de las observaciones se encuentra en ese valor o por debajo.
TPS	Transacciones o solicitudes por segundo, según la unidad definida para la prueba.
SUS	*System Usability Scale*: cuestionario estándar de 10 ítems para estimar usabilidad percibida.
E2E	*End-to-End: prueba que recorre un flujo completo de usuario a través de la interfaz, la API y la persistencia en un entorno integrado.*
Cypress	*Herramienta de pruebas end-to-end para aplicaciones web que ejecuta los escenarios en un navegador y conserva capturas y videos de los fallos.*
Conciliación de inventario	*Comparación entre el stock de apertura, el conteo de botellas sobrantes y los combos descontados para verificar que no exista diferencia residual.*
Plan Maestro	*Este SQAP v3.0: marco normativo congelado que rige la calidad de todos los sprints sin registrar resultados de ejecución.*
Informe de Cierre de Sprint	*Documento independiente (docs/pruebas/informe-sprintX.md) que registra los resultados reales, evidencias y actas de cada iteración.*
PSR-12	*Guía de estilo de codificación extendida de PHP-FIG, verificada en este proyecto con Laravel Pint.*
Husky	*Herramienta para definir hooks de Git; aquí ejecuta la verificación de Laravel Pint en pre-commit.*
TDD	*Test-Driven Development*: desarrollo guiado por pruebas, en el que se crea una prueba antes de implementar la solución correspondiente.
SAST	*Static Application Security Testing*: análisis estático del código orientado a detectar problemas de seguridad.
Quality Gate	Conjunto de condiciones automatizadas que determina si una versión satisface los umbrales de calidad configurados.
PR	*Pull Request*: propuesta de cambio revisable antes de integrarse en una rama.
RACI	Matriz de responsabilidades: Responsable, Aprobador/Accountable, Consultado e Informado.
Defecto S1	Incidencia de severidad crítica según la matriz interna del proyecto.
Idempotencia	Propiedad por la que repetir una solicitud con la misma identidad no produce efectos adicionales no deseados.
Trazabilidad	Capacidad de relacionar requisito, cambio de código, prueba, defecto y evidencia.
CI/CD	Integración continua y entrega/despliegue continuo mediante procesos automatizados.
Laravel Pint	Herramienta de formato de código PHP utilizada para aplicar convenciones de estilo.
Pest	Framework de pruebas para PHP que puede utilizar PHPUnit como base de ejecución.

16. Procedimiento de cambio e historia del SQAP

16.1 Solicitud y aprobación de cambios

Cualquier modificación de este SQAP debe realizarse mediante Pull Request sobre `docs/calidad/SQAP-Maestro-v3.0.docx` o la ruta oficial definida por el repositorio.

Flujo obligatorio:

1. Registrar el motivo, la sección afectada, el impacto en métricas, pruebas, roles y riesgos.
2. Crear una rama de cambio y modificar el documento.
3. Actualizar cualquier configuración automatizada afectada para mantener la homologación entre el umbral escrito y el ejecutable.

4. Revisar las consecuencias sobre el alcance de los sprints y sus criterios de aceptación.
5. Obtener revisión de una persona distinta del autor.
6. Actualizar el historial de versiones y enlazar el PR/acta.
7. Integrar el cambio solo después de la aprobación y de los chequeos obligatorios.

Una reducción de umbral no puede utilizarse únicamente para hacer pasar el pipeline. Debe incluir justificación técnica, impacto, riesgo residual y aprobación documentada. Los cambios que alteren las 17 historias o los 79 puntos de la línea base deben tratarse además como cambios de alcance del proyecto.

Congelamiento del Plan Maestro: una vez aprobada, esta versión 3.0 permanece congelada. Solo se modifica por un cambio normativo (umbral, estándar, herramienta, rol o alcance del Backlog) conforme a este procedimiento. Los resultados de ejecución de los Sprints 1 a 4 y de cualquier iteración posterior no son causa de modificación: se registran en los Informes de Cierre de Sprint (sección 4.1). Todo cambio normativo genera una nueva versión del plan y su fila en el historial.

16.2 Historial de versiones

Versión	Fecha	Cambio	Autor	Revisor
0.1	04/08/2026	Estructura inicial del plan.	André	Félix
0.5	18/08/2026	Primer conjunto de métricas, prácticas y herramientas.	André	Félix
0.9	01/09/2026	Borrador consolidado para revisión.	EQ-LAB07	Revisión cruzada
0.9.1	06/10/2026	Ajuste de umbrales y asignación de roles.	André	Félix
1.0	13/10/2026	Consolidación para el Sprint 1 de Discoteca Tío Chu.	EQ-LAB07	Aprobación formal pendiente de evidencia
2.0	20/10/2026	Actualización para el Sprint 2 (h14, h13, h6, h4): alcance, criticidad, métricas M-02 y M-07 a M-15, Cypress en pruebas E2E, casos de prueba, riesgos, anexos y referencias.	EQ-LAB07	Pendiente de revisión cruzada y aprobación formal
3.0	29/10/2026	Reestructuración a Plan Maestro congelado para los Sprints 1 a 4: matriz de 17 historias (Backlog v5), métricas permanentes, PSR-12 con Pint y Husky, E2E con Cypress en el 100 % de las historias y desacoplamiento de evidencias en Informes de Cierre de Sprint.	EQ-LAB07	Pendiente de revisión cruzada y aprobación formal

ANEXO 1. Reporte de defecto crítico S1

> *Nota: Caso demostrativo para validar el procedimiento. Requiere reproducción para ser un incidente real..*

DEF-2026-001 — Caso demostrativo

ID: DEF-2026-001 | **Historia:** h13 — Retención y Neto de Tarjetas POS
Título: Diferencia en cálculo de comisión y neto de tarjeta POS en h13
Severidad / Prioridad: S1 (Crítica - afecta conciliación) / P1 (Corrección antes de liberar)
Estado: Ejemplo no ejecutado.
Reportado por: Maturano Zárate Félix | **Asignado a:** Clarós André
Fecha: dd/mm/aaaa (fecha de detección)

1. Descripción: El cálculo del neto POS no coincide con la fórmula de comisión aprobada, pudiendo generar cifras de liquidez incorrectas.

2. Precondiciones: Entorno de pruebas activo, tasa documentada y uso de transacciones sintéticas.

3. Pasos para reproducir: Ingresar importe bruto y tasa en formulario POS -> Guardar -> Comparar comisión/neto con el cálculo de referencia.

4. Resultado esperado: La comisión y el neto deben coincidir exactamente con la fórmula aprobada y método de redondeo.

5. Resultado obtenido: No medido (pendiente de ejecución).

6. Evidencia requerida: Captura/JSON, ID de commit, datos sintéticos y log de prueba.

7. Corrección y SLA: Añadir prueba unitaria, corregir lógica, validar PR. Tiempos: Respuesta <2h, Resolución <24h.

ANEXO 2. Autoevaluación contra los siete indicadores de verificabilidad

Evaluador: Maturano Zárate Félix — SQA Secundario / Arquitecto
Documento evaluado: SQAP-Maestro-v3.0
Fecha de revisión prevista: se registra en el Informe de Cierre de cada sprint
Resultado: se registra en el Informe de Cierre de cada sprint; este Plan Maestro no contiene resultados de evaluación.

Indicador	Pregunta de verificación	Evidencia prevista en el SQAP	Registro del resultado
1. Completo	¿Están las 16 secciones y los 3 anexos con contenido específico del proyecto?	Secciones 1–16; anexos 1–3	Informe de Cierre de Sprint: verificación estructural
2. Medible	¿Los objetivos tienen fórmulas o reglas y umbrales numéricos?	Sección 5: cobertura, p95, TPS, seguridad, duplicación, SUS, exactitud funcional y E2E	Informe de Cierre de Sprint: validación de reglas de cálculo y umbrales
3. Instrumentado	¿Cada métrica indica instrumento y condiciones de medición?	Pest/PHPUnit, Cypress, SonarQube, JMeter y casos de prueba	Informe de Cierre de Sprint: instalación, configuración y ejecución de las herramientas
4. Asignado e independiente	¿Hay responsable y revisor distinto del autor?	Sección 3, matriz RACI y regla de aprobación de PR	Informe de Cierre de Sprint; la independencia organizativa está limitada por el tamaño del equipo
5. Consistente	¿Los valores del documento coinciden con las configuraciones y el alcance?	Secciones 1, 5, 7, 9 y 16; historial de cambios	Informe de Cierre de Sprint: homologación con repositorio y CI
6. Trazable	¿Cada requisito puede relacionarse con pruebas, cambios y defectos?	IDs `h2` a `h18` (17 historias, matriz 1.1); casos TC-Hn-nn y PR/Issues	Informe de Cierre de Sprint: enlaces concretos a pruebas, PR e Issues
7. Evidenciable	¿Los resultados pueden comprobarse con artefactos, logs y actas?	Sección 12 y anexos; reportes CI/JMeter/Cypress	Informe de Cierre de Sprint: artefactos, logs y actas adjuntos

Lista de comprobación de cierre (plantilla que se completa en el Informe de Cierre de cada sprint)

- [ ] SRS y SAD vigentes aprobados; historias del sprint trazadas a pruebas.
- [ ] Cobertura, análisis estático y seguridad ejecutados/configurados.
- [ ] Prueba JMeter ejecutada (archivos .jmx, .jtl generados).
- [ ] Escenarios E2E de Cypress ejecutados en CI; reporte, capturas y videos archivados.
- [ ] PRs de André aprobados por Félix (revisión cruzada).
- [ ] Cero defectos S1/S2 abiertos.
- [ ] Backups comprobados; informe final documentado.

ANEXO 3. Análisis escrito y respuestas a preguntas de reflexión

3.1 Usabilidad en cada sprint: Se evaluará combinando la observación de tareas clave (ej. apertura de sesión, cobro QR, liquidación por turno, pago de jornales, cierre de noche) sin errores evitables, junto con el cuestionario SUS (meta ≥68/100).

3.2 Inspección humana vs CI/CD: CI/CD detecta regresiones y fallos técnicos repetibles; la inspección humana es vital para entender intenciones de negocio, fórmulas, ambigüedades y diseño que la automatización ignora.

3.3 Consecuencias de bajar umbrales: Oculta defectos reales (ej. fallos financieros) y daña la confianza. Éticamente es engañoso; cualquier cambio de umbral requiere justificación técnica y análisis de riesgo, no solo para "hacer pasar" el pipeline.

3.4 Independencia SQA en equipo de dos: Se garantiza operativamente mediante revisión cruzada de Pull Requests (el autor no aprueba su propio código) y decisiones conjuntas en defectos críticos.

3.5 Significado de un "plan verificable": Significa depender de evidencias concretas y no de afirmaciones vagas. Requiere umbrales, herramientas y responsables (ej. "p95 ≤ 1.5s en JMeter" en lugar de "el sistema es rápido").

Criterio final de aceptación del SQAP y de cada Sprint

- **Del SQAP (Documento):** Aprobado al revisar todas las secciones/anexos, resolver observaciones y asegurar que los umbrales sean medibles. *(Aprobar el plan no aprueba el software). Una vez aprobado, el Plan Maestro queda congelado (sección 16).*
- **De cada Sprint (Release):**
  1. Historias del sprint verificadas y trazadas a casos de prueba.
  2. Pruebas obligatorias (unitarias, integración, E2E con Cypress y carga) ejecutadas con evidencia archivada.
  3. Cobertura, calidad estática (incluido PSR-12 con Laravel Pint) y rendimiento (JMeter) medidos y dentro de los umbrales.
  4. Cero defectos críticos (S1/S2) abiertos.
  5. Todo cambio integrado tiene revisión independiente (PRs).
  6. Desviaciones y riesgos declarados en el Informe de Cierre del sprint.

Referencias bibliográficas

- IEEE. *IEEE Std 730-2002: Standard for Software Quality Assurance Plans*. IEEE, 2002.
- IEEE. *IEEE Std 1028-2008: IEEE Standard for Software Reviews and Audits*. IEEE, 2008.
- ISO/IEC. *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. ISO, 2011.
- PHP Framework Interop Group (PHP-FIG). *PSR-12: Extended Coding Style Guide*.
- Laravel. *Laravel Documentation*, versión compatible con Laravel 13.
- Pest PHP. *Pest Documentation*.
- Laravel. *Laravel Pint Documentation*.
- Husky. *Husky Documentation*.
- Apache Software Foundation. *Apache JMeter User Manual*, versión 5.6.
- Cypress.io. *Cypress Documentation*.
- EQ-LAB07. *Historias_Usuario_Tio_Chu_v5_4_Sprints.xlsx*. Pila de producto, plan de sprints y dependencias de las 17 historias del proyecto.